EECE 2311

Lab for EECE2310: Introduction to Digital Design and Computer Architecture

SPRING 2026

Lab Assignment #04

John Kimani

EECE2311 Lab Section # 33858

Sijing Deng

Date Submitted:

Mar 09 2026

Abstract

The purpose of this lab is to design an arithmetic-logic unit (ALU) that operates four different operations: addition, subtraction, logic AND, and logic OR. on 4-bit binary numbers.

Introduction

The purpose of this lab is to design an arithmetic-logic unit (ALU) that operates on 4-bit binary numbers. The ALU will support four different operations: addition, subtraction, logic AND, and logic OR. The results will be displayed in binary format on the LEDs as well as in decimal format on the 7-Segment displays. You are not allowed to use any of the Quartus library modules unless for the modules that are explicitly allowed in the lab instructions. a digit 0 to 9 and lights up the desired segments to display the corresponding digit. You are not allowed to use any of the Quartus software library modules unless for the modules that are explicitly allowed in the lab instructions.

Lab Setup

Pre-Lab

An unsigned 4-bit binary number has values that range from 0 to 15. So, to display its decimal value, we need two 7-segments displays. But first we need to convert the 4-bit binary value to its binary-coded decimal (BCD) format as shown in table 1.

In this lab you will need to implement a digital circuit that takes as an input the binary value, $B_3B_2B_1B_0$, and generates as an output its corresponding BCD value, $D_4D_3D_2D_1D_0$. For this pre-lab assignment, use five Karnaugh maps, based on the shown truth table, to find the minimized Boolean expression of each output bit ($D_4D_3D_2D_1D_0$) as a function of the input bits ($B_3B_2B_1B_0$).

Binary B_3 B_2 B_1 B_0	2-Digit Decimal	BCD D_4 D_3 D_2 D_1 D_0
0 0 0 0	00	0 0 0 0 0
0 0 0 1	01	0 0 0 0 1
0 0 1 0	02	0 0 0 1 0
0 0 1 1	03	0 0 0 1 1
0 1 0 0	04	0 0 1 0 0
0 1 0 1	05	0 0 1 0 1
0 1 1 0	06	0 0 1 1 0
0 1 1 1	07	0 0 1 1 1
1 0 0 0	08	0 1 0 0 0
1 0 0 1	09	0 1 0 0 1
1 0 1 0	10	1 0 0 0 0
1 0 1 1	11	1 0 0 0 1
1 1 0 0	12	1 0 0 1 0
1 1 0 1	13	1 0 0 1 1
1 1 1 0	14	1 0 1 0 0
1 1 1 1	15	1 0 1 0 1

Table 1: D4D3D2D1D0 Combined Truth Table

Equipments

- DE1-SoC FPGA development board
- Computer with Intel Quartus Prime soRware installed
- USB programming cable for FPGA configuraNon
- Previously designed half-adder and full-adder schemaNc files
- ConnecNng cables and power supply for the board

Binary $B_3\ B_2\ B_1\ B_0$	2-Digit Decimal	D_4
0 0 0 0	00	0
0 0 0 1	01	0
0 0 1 0	02	0
0 0 1 1	03	0
0 1 0 0	04	0
0 1 0 1	05	0
0 1 1 0	06	0
0 1 1 1	07	0
1 0 0 0	08	0
1 0 0 1	09	0
1 0 1 0	10	1
1 0 1 1	11	1
1 1 0 0	12	1
1 1 0 1	13	1
1 1 1 0	14	1
1 1 1 1	15	1

Binary $B_3\ B_2\ B_1\ B_0$	2-Digit Decimal	D_3
0 0 0 0	00	0
0 0 0 1	01	0
0 0 1 0	02	0
0 0 1 1	03	0
0 1 0 0	04	0
0 1 0 1	05	0
0 1 1 0	06	0
0 1 1 1	07	0
1 0 0 0	08	1
1 0 0 1	09	1
1 0 1 0	10	0
1 0 1 1	11	0
1 1 0 0	12	0
1 1 0 1	13	0
1 1 1 0	14	0
1 1 1 1	15	0

Binary $B_3\ B_2\ B_1\ B_0$	2-Digit Decimal	D_2
0 0 0 0	00	0
0 0 0 1	01	0
0 0 1 0	02	0
0 0 1 1	03	0
0 1 0 0	04	1
0 1 0 1	05	1
0 1 1 0	06	1
0 1 1 1	07	1
1 0 0 0	08	0
1 0 0 1	09	0
1 0 1 0	10	0
1 0 1 1	11	0
1 1 0 0	12	0
1 1 0 1	13	0
1 1 1 0	14	1
1 1 1 1	15	1

Binary $B_3\ B_2\ B_1\ B_0$	2-Digit Decimal	D_1
0 0 0 0	00	0
0 0 0 1	01	0
0 0 1 0	02	1
0 0 1 1	03	1
0 1 0 0	04	0
0 1 0 1	05	0
0 1 1 0	06	1
0 1 1 1	07	1
1 0 0 0	08	0
1 0 0 1	09	0
1 0 1 0	10	0
1 0 1 1	11	0
1 1 0 0	12	1
1 1 0 1	13	1
1 1 1 0	14	0
1 1 1 1	15	0

Binary $B_3\ B_2\ B_1\ B_0$	2-Digit Decimal	D_0
0 0 0 0	00	0
0 0 0 1	01	1
0 0 1 0	02	0
0 0 1 1	03	1
0 1 0 0	04	0
0 1 0 1	05	1
0 1 1 0	06	0
0 1 1 1	07	1
1 0 0 0	08	0
1 0 0 1	09	1
1 0 1 0	10	0
1 0 1 1	11	1
1 1 0 0	12	0
1 1 0 1	13	1
1 1 1 0	14	0
1 1 1 1	15	1

D_4 D_3 D2 D_1 D_0

Table 2: Truth Table for D_4, D_3, D_2, D_1, D_0

B_3B_2/B_1B_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	0	1	1

Table 3: k-map for D_4

$D_4 = B_3B_2 + B_3B_1$

B_3B_2/B_1B_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	1	1	0	0

Table 4: k-map for D_3

$D_3 = B_3B_2'B_1'$

B_3B_2/B_1B_0	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	1	1
10	0	0	0	0

Table 5: k-map for D_2

$D_2 = B_3'B_2 + B_2B_1$

B_3B_2/B_1B_0	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	1	1	0	0
10	0	0	0	0

Table 6: k-map for D_1

$D_1 = B_3'B_1 + B_3B_2B_1'$

B_3B_2/B_1B_0	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

Table 7: k-map for D_0

$D_0 = B_0$

Boolean Expressions:

$D_4 = B_3B_2 + B_3B_1$
$D_3 = B_3B_2'B_1'$
$D_2 = B_3'B_2 + B_2B_1$
$D_1 = B_3'B_1 + B_3B_2B_1'$
$D_0 = B_0$

Binary to BCD Decoder

For the 4-bit inputs, $B_3B_2B_1B_0$, use the first four switches: SW[3], W[2], SW[1], and SW[0]. For the 5-bit output, D4D3D2D1D0 , connect them to the first five LEDs: LEDR[4] , LEDR[3], LEDR[2], LEDR[1], and LEDR[0]. Follow the same procedure outlined in the Quartus Prime Lite Reference.

B_3	B_2	B_1	B_0
sw[3]	sw[2]	sw[1]	sw[0]

Table 8: Switch Assignment for Inputs B[3:0]

Signal Name	*FPGA Pin No.*	*Description*	*I/O Standard*
SW[0]	PIN_AB12	Slide Switch[0]	3.3V
SW[1]	PIN_AC12	Slide Switch[1]	3.3V
SW[2]	PIN_AF9	Slide Switch[2]	3.3V
SW[3]	PIN_AF10	Slide Switch[3]	3.3V

Table 9: Pin Assignment of Slide Switches

D_4	D_3	D_2	D_1	D_0
LEDR[4]	LEDR[3]	LEDR[2]	LEDR[1]	LEDR[0]

Table 10: Switch Assignment for Outputs D[4:0]

Signal Name	*FPGA Pin No.*	*Description*	*I/O Standard*
LEDR[0]	PIN_V16	LED [0]	3.3V
LEDR[1]	PIN_W16	LED [1]	3.3V
LEDR[2]	PIN_V17	LED [2]	3.3V
LEDR[3]	PIN_V18	LED [3]	3.3V
LEDR[4]	PIN_W17	LED [4]	3.3V

Table 11: Pin Assignment of LEDs

BINARY TO BCD DECODER SCHEMATIC

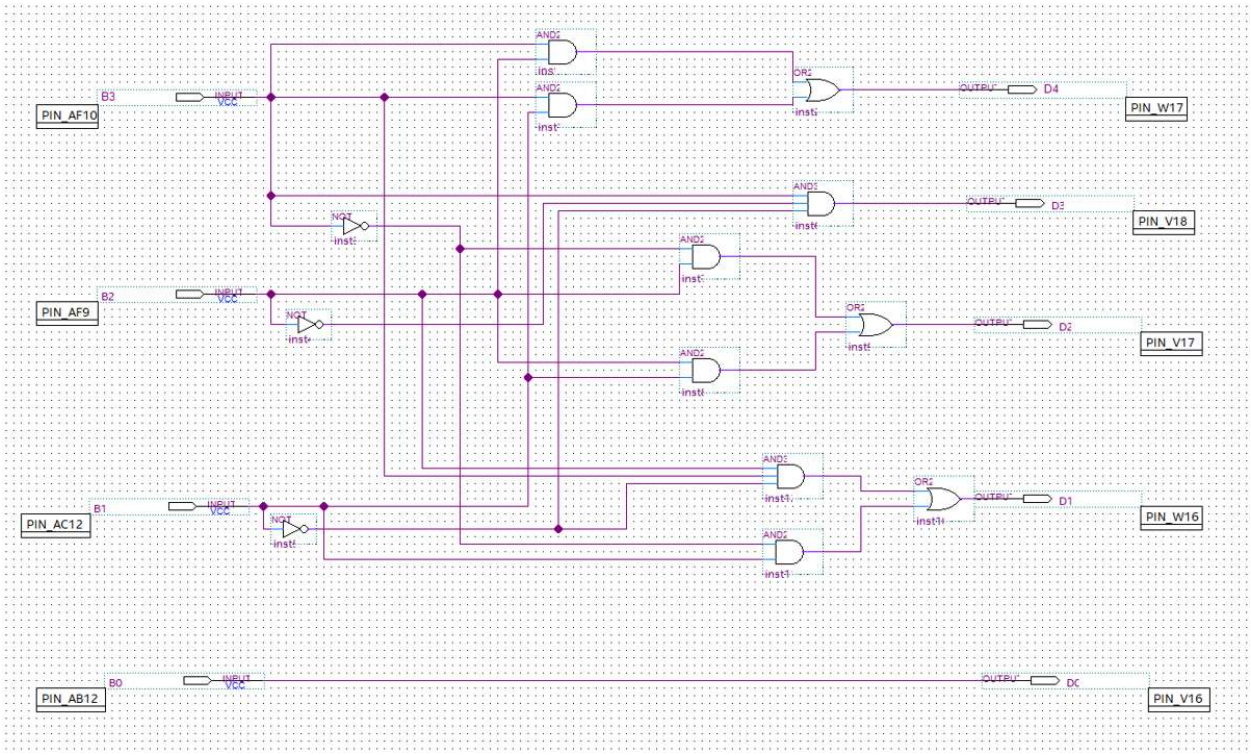


Figure 1: Binary to BCD Decoder Schematic

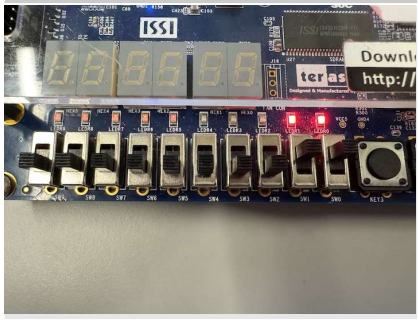
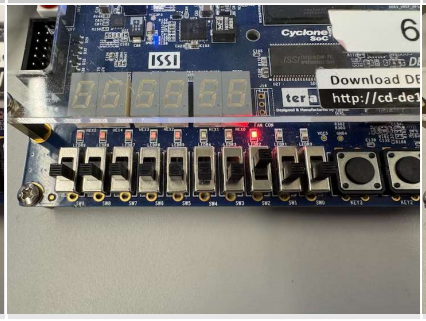
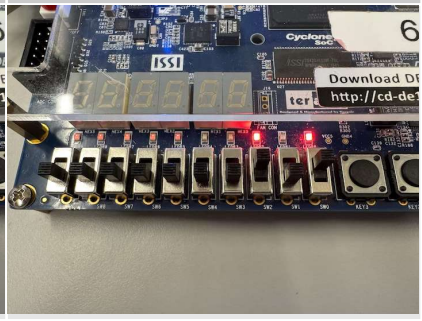
Inputs: $B_3B_2B_1B_0$(0011) Outputs: $D_4D_3D_2D_1D_0$(00011) 2-Digit Decimal: 03	Inputs: $B_3B_2B_1B_0$(0100) Outputs: $D_4D_3D_2D_1D_0$(00100) 2-Digit Decimal: 04	Inputs: $B_3B_2B_1B_0$(0101) Outputs: $D_4D_3D_2D_1D_0$(00101) 2-Digit Decimal: 04
		

Table 12: Display of 2-digital decimal number 03, 04, 05

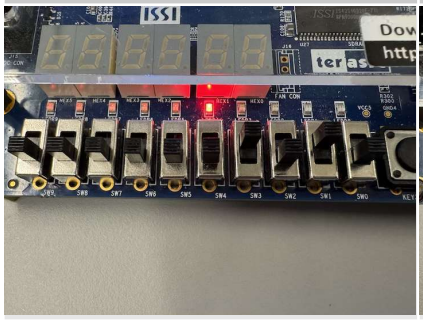
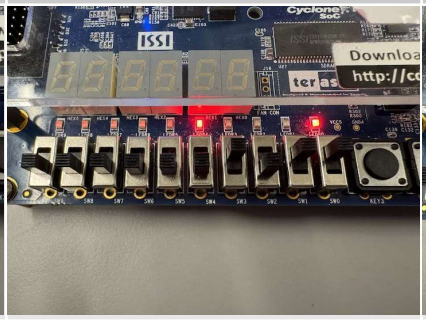
Inputs: $B_3B_2B_1B_0$(1010) Outputs: $D_4D_3D_2D_1D_0$(10000) 2-Digit Decimal: 10	Inputs: $B_3B_2B_1B_0$(1011) Outputs: $D_4D_3D_2D_1D_0$(10001) 2-Digit Decimal: 11	Inputs: $B_3B_2B_1B_0$(1100) Outputs: $D_4D_3D_2D_1D_0$(10010) 2-Digit Decimal: 12
		

Table 13: Display of 2-digital decimal number 10, 11, 12

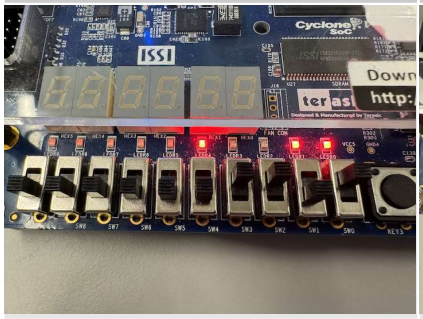
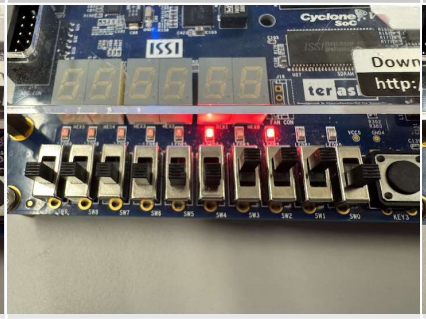
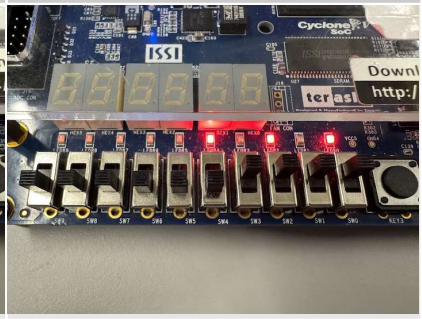
Inputs: $B_3B_2B_1B_0$(1101) Outputs: $D_4D_3D_2D_1D_0$(10011) 2-Digit Decimal: 13	Inputs: $B_3B_2B_1B_0$(1110) Outputs: $D_4D_3D_2D_1D_0$(10100) 2-Digit Decimal: 14	Inputs: $B_3B_2B_1B_0$(1111) Outputs: $D_4D_3D_2D_1D_0$(10101) 2-Digit Decimal: 15
		

Table 14: Display of 2-digital decimal number 13, 14, 15

THE 4-BIT ALU IMPLEMENTATION PIN ASSIGNMENT

SrcA[0]	SrcA[1]	SrcA[2]	SrcA[3]
sw[0]	sw[1]	sw[2]	sw[3]

Table 15: Switch Assignment for SrcA[3..0]

SrcB[0]	SrcB[1]	SrcB[2]	SrcB[3]
sw[4]	sw[5]	sw[6]	sw[7]

Table 16: Switch Assignment for SrcB[3..0]

Signal Name	FPGA Pin No.	Description	I/O Standard
SW[0]	PIN_AB12	Slide Switch[0]	3.3V
SW[1]	PIN_AC12	Slide Switch[1]	3.3V
SW[2]	PIN_AF9	Slide Switch[2]	3.3V
SW[3]	PIN_AF10	Slide Switch[3]	3.3V
SW[4]	PIN_AD11	Slide Switch[4]	3.3V
SW[5]	PIN_AD12	Slide Switch[5]	3.3V
SW[6]	PIN_AE11	Slide Switch[6]	3.3V
SW[7]	PIN_AC9	Slide Switch[7]	3.3V

Table 17: Pin Assignment of Slide Switches sw[7:0]

ALUControl[0]	ALUControl[1]
sw[8]	sw[9]

Table 18: Switch Assignment for SrcB[3..0]

SW[8]	PIN_AD10	Slide Switch[8]	3.3V
SW[9]	PIN_AE12	Slide Switch[9]	3.3V

Table 19: Pin Assignment of Slide Switches sw[9:8]

Result[3]	Result[2]	Result[1]	Result[0]	Overflow
LEDR[3]	LEDR[2]	LEDR[1]	LEDR[0]	LEDR[9]

Table 20: Switch Assignment for Result Outputs [3:0]

Signal Name	FPGA Pin No.	Description	I/O Standard
LEDR[0]	PIN_V16	LED [0]	3.3V
LEDR[1]	PIN_W16	LED [1]	3.3V
LEDR[2]	PIN_V17	LED [2]	3.3V
LEDR[3]	PIN_V18	LED [3]	3.3V
LEDR[9]	PIN_Y21	LED [9]	3.3V

Table 21: Pin Assignment of LEDs

THE 4-BIT ALU IMPLEMENTATION SCHEMATIC

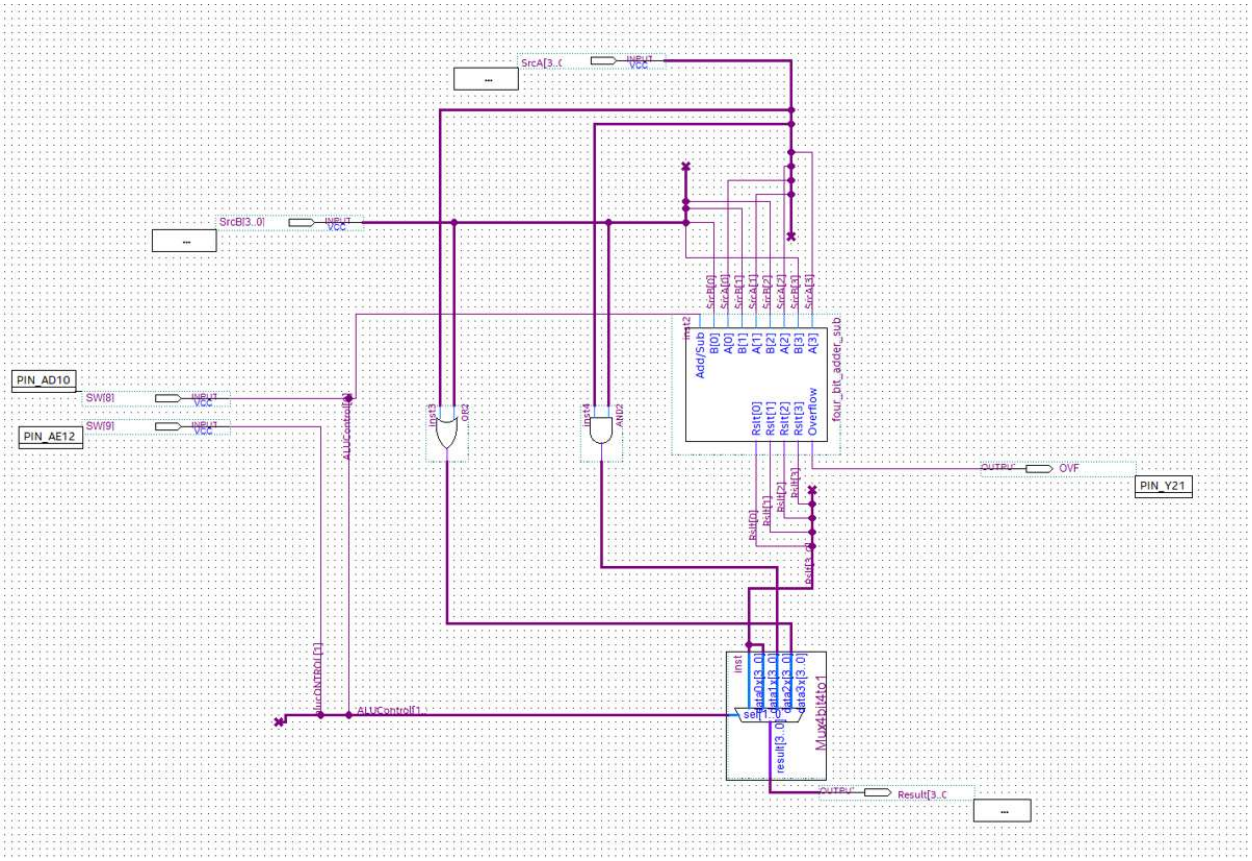


Figure 2: The 4-bit ALU Implementation schematic

#	Operation	ALUControl (SW	SrcA (Decimal)	SrcB (Decimal)	Expected Result	OVF (LEDR[9])	Logic Detail
1	ADD	0	3 (0011)	2 (0010)	0101 (5)	0	Within range.
2	ADD	0	5 (0101)	4 (0100)	1001 (−7)	1	Overflow: 5+4=9, exceeds +7.
3	ADD	0	−4 (1100)	−5 (1011)	0111 (7)	1	Overflow: −4−5=−9, below −8.
4	ADD	0	−1 (1111)	1 (0001)	0000 (0)	0	Standard 2's complement sum.
5	SUB	1	7 (0111)	2 (0010)	0101 (5)	0	7−2=5.
6	SUB	1	5 (0101)	−4 (1100)	1001 (−7)	1	Overflow: 5−(−4)=9.
7	SUB	1	−6 (1010)	3 (0011)	0111 (7)	1	Overflow: −6−3=−9.
8	SUB	1	−2 (1110)	−2 (1110)	0000 (0)	0	−2−(−2)=0.
9	AND	10	−1 (1111)	7 (0111)	0111 (7)	0	Bitwise logic.
10	AND	10	−8 (1000)	7 (0111)	0000 (0)	0	No common bits.
11	OR	11	2 (0010)	4 (0100)	0110 (6)	0	Bitwise logic.
12	OR	11	−1 (1111)	0 (0000)	1111 (−1)	0	Identity.

Table 22: Test bench

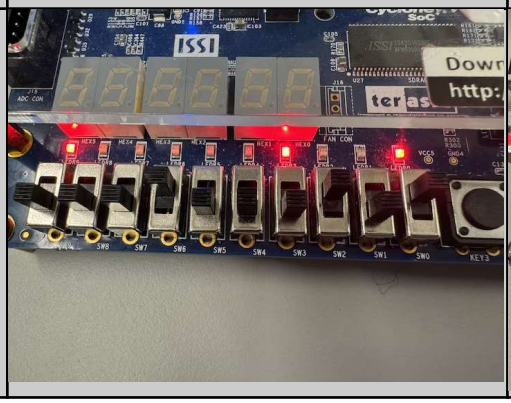
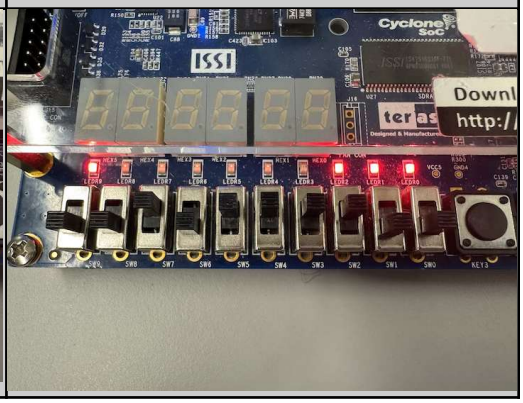
Operation	ADD	ADD
ALUControl (SW[9:8])	00	00
SrcA (SW[3:0])	0101	1100
SrcA (Decimal)	5	-4
SrcB (SW[7:4])	0100	1011
SrcB (Decimal)	4	-5
Expected Result	1001 (9 overflow -> -7)	0111 (-9 overflow -> 7)
OVF (LEDR[9])-	1	1
		

Table 23: Tests for AND

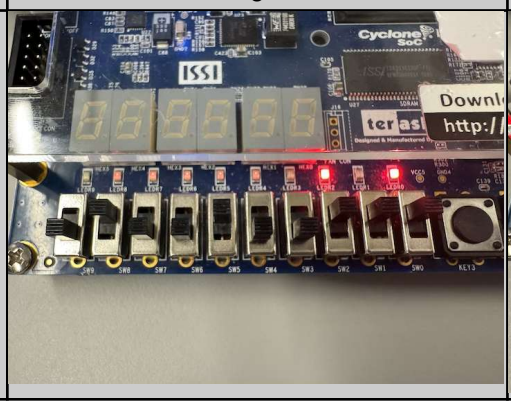
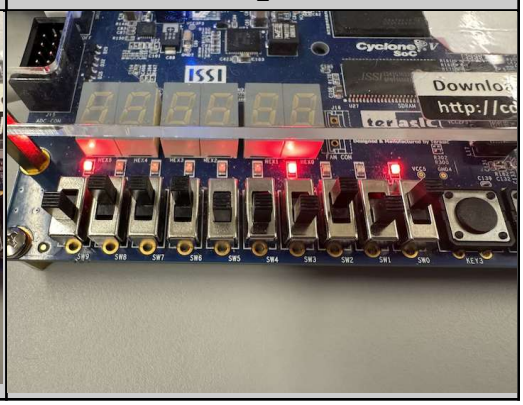
Operation	SUB	SUB
ALUControl (SW[9:8])	01	01
SrcA (SW[3:0])	0111	0101
SrcA (Decimal)	7	5
SrcB (SW[7:4])	0010	1100
SrcB (Decimal)	2	-4
Expected Result	0101 (5)	1001 (9 overflow -> -7)
OVF (LEDR[9])-	0	1
		

Table 24: Tests for SUB

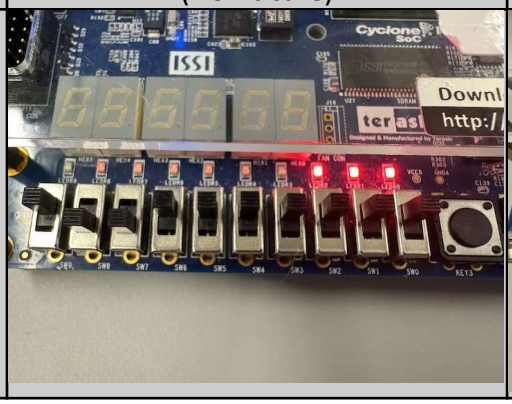
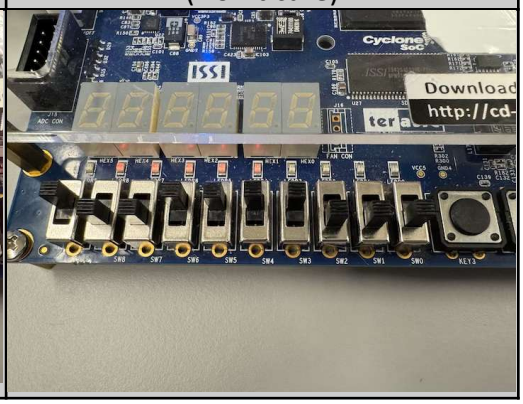
Operation	AND	AND
ALUControl (SW[9:8])	10	10
SrcA (SW[3:0])	1111	1000
SrcA (Decimal)	-1	-8
SrcB (SW[7:4])	0111	0111
SrcB (Decimal)	7	7
Expected Result	0111 (7)	0000 (0) No common bits
OVF (LEDR[9])-	(Don't care)	(Don't care)
		

Table 25: Tests for AND

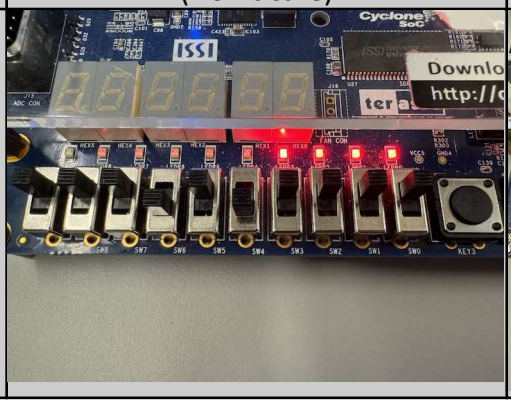
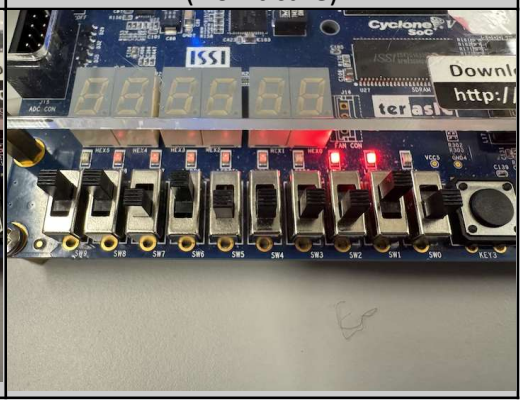
Operation	OR	OR
ALUControl (SW[9:8])	11	11
SrcA (SW[3:0])	1111	0010
SrcA (Decimal)	-1	2
SrcB (SW[7:4])	1010	0100
SrcB (Decimal)	-6	4
Expected Result	1111	0110
OVF (LEDR[9])-	(Don't care)	(Don't care)
		

Table 26: Tests for OR

The ALU Decimal Outputs on the 7-Segment Displays

Extend the Four_Bit_ALU to display the decimal value of its output on the 7-Segment Displays using two of the displays: HEX0 and HEX1 using the decoder.

ALU Operation	Expected Decimal Output on HEX1 HEX0
ADD or SUB (with no overflow)	-8, -7, -6, -5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7
ADD or SUB (with overflow)	88
AND or OR	00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15

Table 27: Tests for OR 7-segments expected outputs

Circuits Inputs				Circuits Outputs				
$ALUControl_1$	$ALUControl_0$	OVF	$Result_3$	$HEX1_{En}$	$HEX1_{Ng}$	$HEX1_S$	$HEX0_S[1]$	$HEX0_S[0]$
A	B	C	D	V	W	X	Y	Z
0	0	0	0	0	0	x	0	0
0	0	0	1	1	1	x	0	1
0	0	1	x	1	0	0	1	0
0	1	0	0	0	0	x	0	0
0	1	0	1	1	1	x	0	1
0	1	1	x	1	0	0	1	0
1	0	x	x	1	0	1	1	1
1	1	x	x	1	0	1	1	1

Table 28: Truth tables of the 7-segments control circuits

ALUControl[0]	ALUControl[1]
sw[8]	sw[9]

Table 29: Switch Assignment for SrcB[3..0]

A	B	C	D	HEX1en (V)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Table 30: truth table for HEX1en (V)

AB/ CD	00	01	11	10
00	0	1	1	1
01	0	1	1	1
11	1	1	1	1
10	1	1	1	1

Table 31: K-map for HEX1en (V)
V = A + C + D

A	B	C	D	HEX1ng (W)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Table 32: truth table for HEX1ng (W)

AB/ CD	00	01	11	10
00	0	1	0	0
01	0	1	0	0
11	0	0	0	0
10	0	0	0	0

Table 33: K-map for HEX1ng (W)
W= A'C'D

A	B	C	D	HEX1s (X)
0	0	0	0	x
0	0	0	1	x
0	0	1	0	0
0	0	1	1	0
0	1	0	0	x
0	1	0	1	x
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Table 34: truth table for HEX1s (X)

AB/ CD	00	01	11	10
00	x	x	0	0
01	x	x	0	0
11	1	1	1	1
10	1	1	1	1

Table 35: K-map for HEX1s (X)
X = A

A	B	C	D	HEX0S[1] (Y)
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Table 36: truth table for HEX0S[1] (Y)

AB/ CD	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	1	1	1	1
10	1	1	1	1

Table 37: K-map for HEX0S[1] (Y)
Y = A + C

A	B	C	D	HEX0S[0] (Z)
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Table 38: truth table for HEX0S[0] (Z)

AB/ CD	00	01	11	10
00	0	1	0	0
01	0	1	0	0
11	1	1	1	1
10	1	1	1	1

Table 39: K-map for HEX0S[0] (Z)
Z = A+C'D

Boolean Expressions for V, W, X, Y, Z

V = A + C + D
W= A'C'D
X = A
Y = A + C
Z = A+C'D

Schematic for ALU Decimal Outputs on the 7-Segment Displays

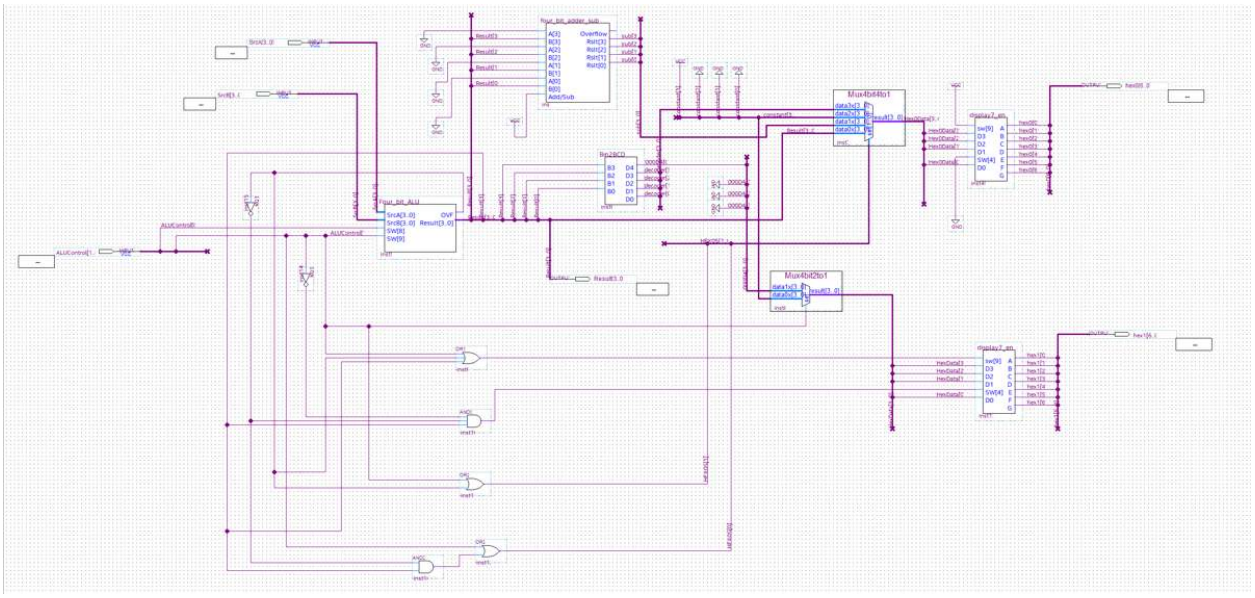


Figure 3: ALU Decimal Outputs on the 7-Segment schematic

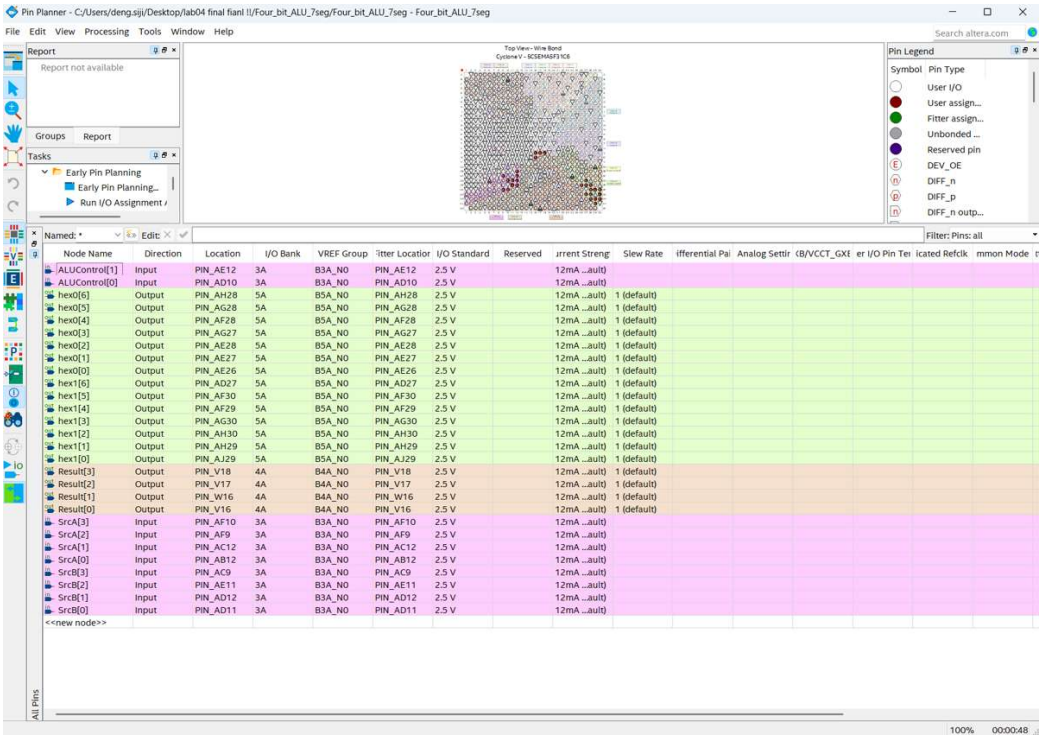


Figure 4: ALU Decimal Outputs on the 7-Segment Pin Assignments

RESULT

Addition (ALUControl = 00)

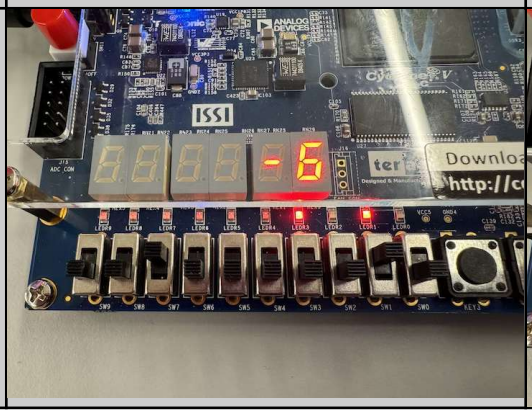
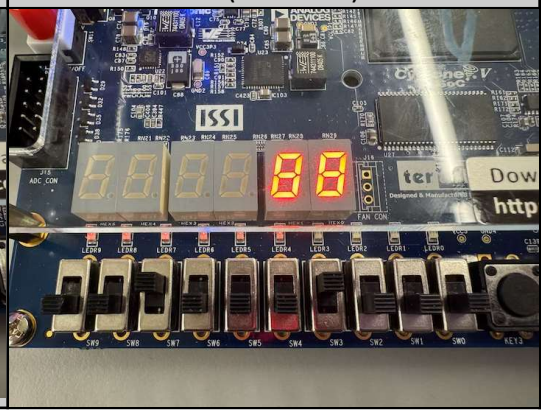
Operation	ADD	ADD
ALUControl (SW[9:8])	00	00
SrcA (SW[3:0])	0010	1000
SrcA (Decimal)	2	-8
SrcB (SW[7:4])	1000	1000
SrcB (Decimal)	-8	-8
Expected Result	1010	(1)0000 (-16 overflow)
OVF (LEDR[9])-	-6	88 (overflow)
		

Table 40: Tests for AND

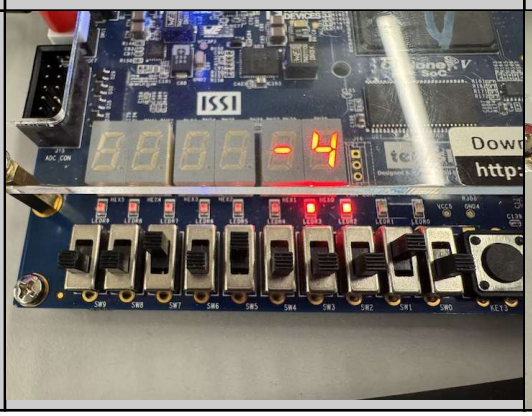
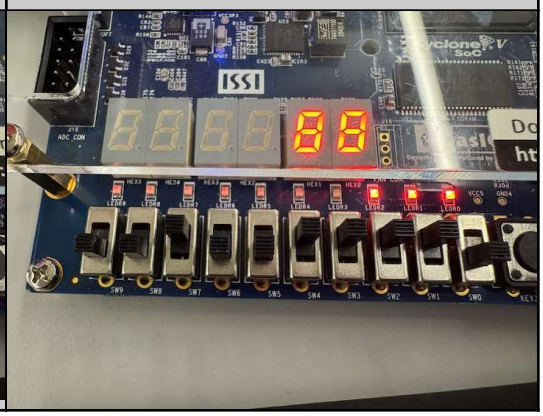
Operation	ADD	ADD
ALUControl (SW[9:8])	00	00
SrcA (SW[3:0])	0010	1110
SrcA (Decimal)	2	-2
SrcB (SW[7:4])	1010	1001
SrcB (Decimal)	-6	-7
Expected Result	1100	(1)0111
OVF (LEDR[9])-	-4	88
		

Table 41: Tests for AND

RESULT

Subtraction (ALUControl = 01)

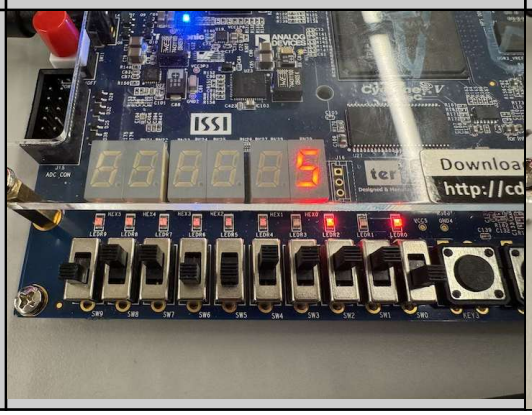
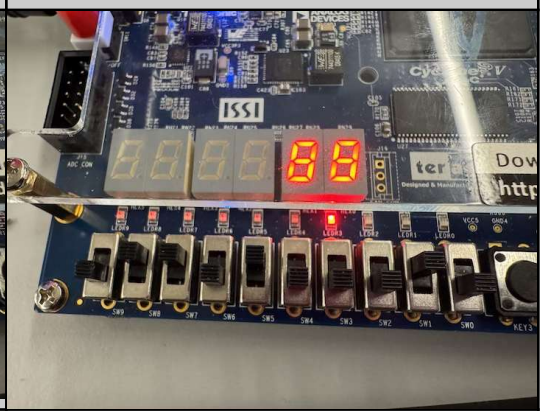
Operation	SUB	SUB
ALUControl (SW[9:8])	01	01
SrcA (SW[3:0])	1110	0010
SrcA (Decimal)	-2	2
SrcB (SW[7:4])	1001	1010
SrcB (Decimal)	-7	-6
Expected Result	0101	1000 (8 overflow)
OVF (LEDR[9])-	5	88
		

Table 42: Tests for subtraction

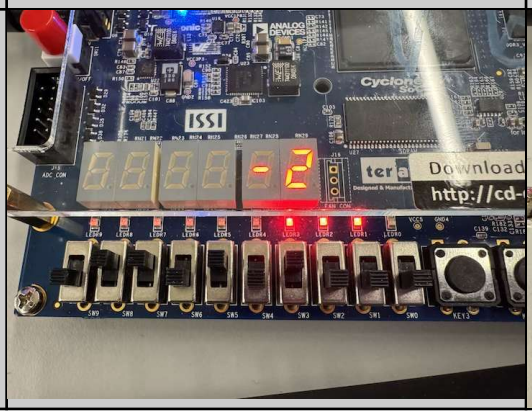
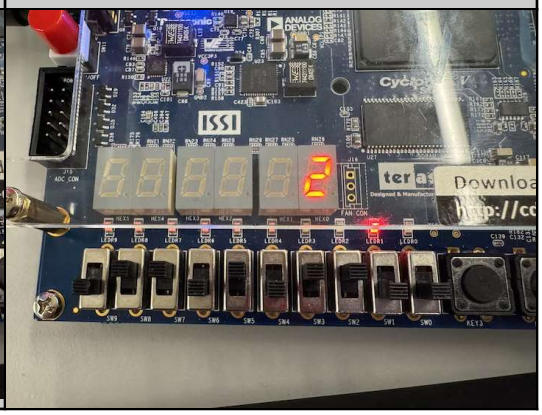
Operation	SUB	SUB
ALUControl (SW[9:8])	01	01
SrcA (SW[3:0])	1000	1100
SrcA (Decimal)	-8	-4
SrcB (SW[7:4])	1010	1010
SrcB (Decimal)	-6	-6
Expected Result	1110	0010
OVF (LEDR[9])-	-2	2
		

Table 43: Tests for subtraction

RESULT

AND (ALUControl = 10)

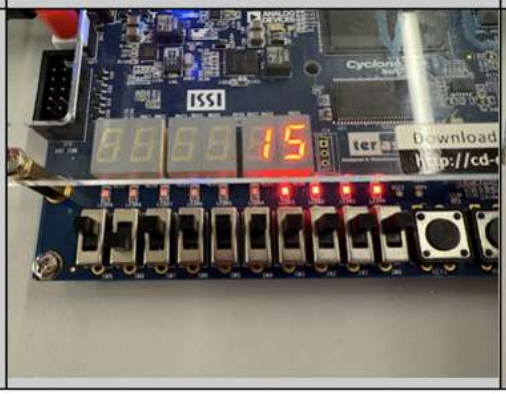
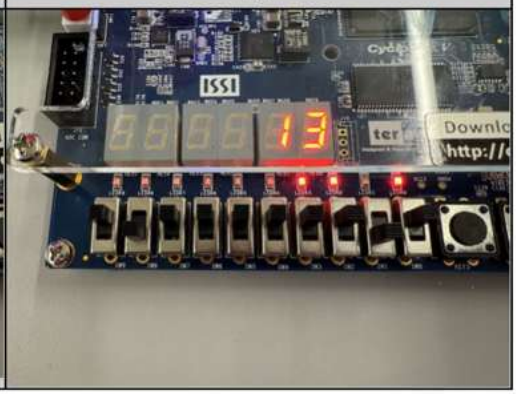
Operation	AND	AND
ALUControl (SW[9:8])	10	10
SrcA (SW[3:0])	1111	1101
SrcA (Decimal)	15	13
SrcB (SW[7:4])	1111	1111
SrcB (Decimal)	15	15
Expected Result	1111	1101
HEX1 HEX0	15	13
		

Table 44: tests for AND

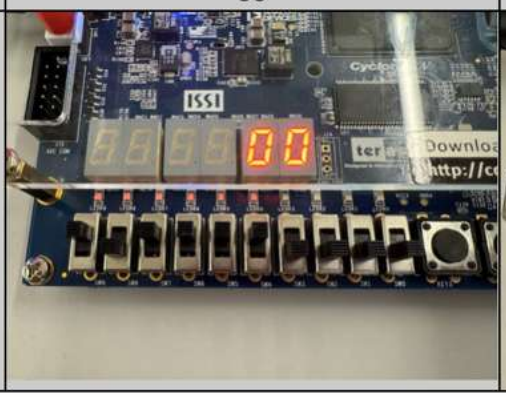
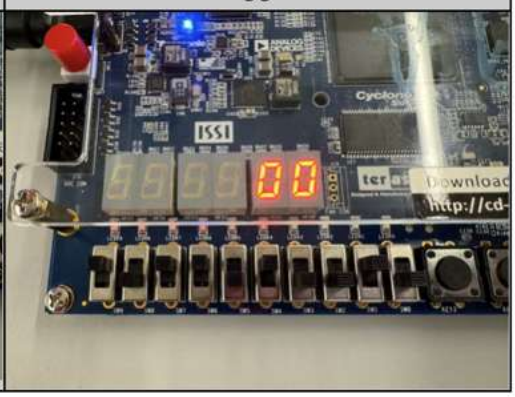
Operation	AND	AND
ALUControl (SW[9:8])	10	10
SrcA (SW[3:0])	0000	0010
SrcA (Decimal)	0	2
SrcB (SW[7:4])	1111	1101
SrcB (Decimal)	15	13
Expected Result	0000	0000
HEX1 HEX0	00	00
		

Table 45: tests for AND

RESULT

OR (ALUControl = 11)

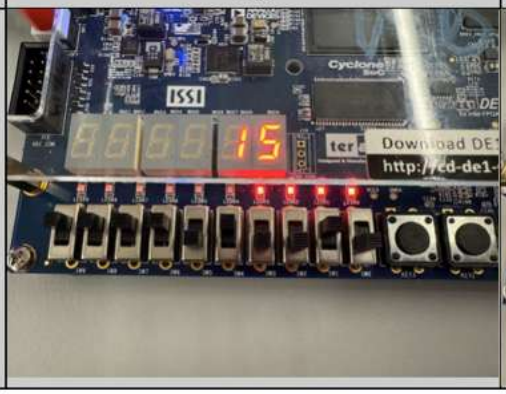
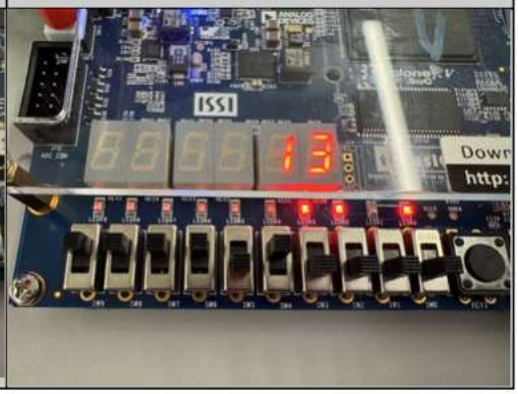
Operation	OR	OR
ALUControl (SW[9:8])	11	11
SrcA (SW[3:0])	0010	0000
SrcA (Decimal)	2	0
SrcB (SW[7:4])	1101	1101
SrcB (Decimal)	13	13
Expected Result	1111	1101
HEX1 HEX0	15	13
		

Table 46: Tests for OR

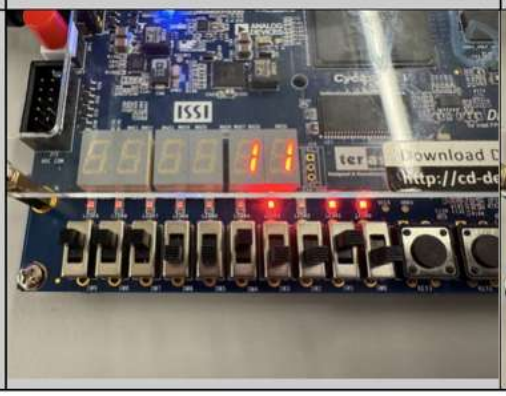
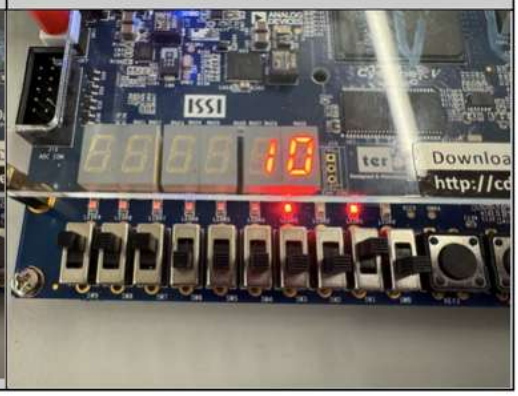
Operation	OR	OR
ALUControl (SW[9:8])	11	11
SrcA (SW[3:0])	0010	0010
SrcA (Decimal)	2	2
SrcB (SW[7:4])	1001	1000
SrcB (Decimal)	9	8
Expected Result	1011	1010
HEX1 HEX0	11	10
		

Table 47: Tests for OR

Results and Analysis

Results

The primary objective of this lab was to design and implement a 4 bit Arithmetic Logic Unit (ALU) with a 7 segment display system. The implementation was verified through several stages:

ALU Operation: The ALU successfully performed addition, subtraction, bitwise AND, and bitwise OR based on the 2 bit ALUControl input.

Binary to BCD Conversion: A decoder was implemented to convert 4 bit binary values into 5 bit BCD format (D4, D3, D2, D1, D0) to drive two decimal displays.

Signed Arithmetic Display: For addition and subtraction, the system correctly identified signed 2's complement results.

Positive results (0 to 7) were displayed on HEX0 while HEX1 remained blank.

Negative results (-1 to -8) correctly triggered the Ng (negative) input on HEX1 to show a minus sign, while the magnitude was displayed on HEX0.

Overflow Detection: In the event of an arithmetic overflow (OVF=1), the control logic successfully overrode the standard BCD output to display "88" across HEX1 and HEX0 as a dedicated error indicator.

Logical Result Display: In AND/OR modes, the system treated results as unsigned values (00 to 15), utilizing both HEX1 and HEX0 to show the full decimal range.

Analysis

The experimental data confirmed that the integrated circuit behaved according to the theoretical Boolean expressions derived from the Karnaugh maps.

Control Logic Integration: The use of the Combinational Circuits block to generate select lines (HEX1 and HEX0) for the output multiplexers showed how status flags like OVF and sign bits like Result3 can dynamically route data.

Magnitude Extraction: To display negative numbers, the analysis showed that the 2's complement of the ALU result must be fed to the BCD decoder to show the absolute value (magnitude) next to the negative sign.

Hardware Constraints: Because the DE1-SoC 7 segment displays require specific activation patterns, the design successfully mapped the 4 bit D inputs through the display7_en decoder to the physical segments.

Priority Logic: The "88" overflow display showed the implementation of priority logic, where the OVF signal acted as a primary control that "masked" the actual mathematical result to prevent the display of incorrect data.

Conclusion

This lab successfully demonstrated the transition from 4 bit binary arithmetic to a human readable decimal interface. By combining a functional ALU with a BCD decoder and custom combinational control logic, we verified that complex digital systems could be managed through modular design. The successful implementation on the DE1-SoC board confirmed that the gate level designs for sign handling and display blanking functioned effectively in a physical environment.

References

1. Michael Benjamin, “Lab Report Guide”, Northeastern University, January 18, 2006.

2. EECE 2311 lab03 pdf.